

Thought Tree Framework

An open framework for cognitive programming with LLMs.

Thought Tree is a framework for describing complex LLM-assisted work as structured, executable, inspectable workflows.

Instead of relying on a single prompt, an opaque agent loop, or an informal chain of model calls, a Thought Tree defines:

- what inputs are required;
- what intermediate artefacts should be produced;
- what operations transform one artefact into another;
- how outputs should be reviewed or validated;
- what final outputs should be returned;
- and what execution trace should be preserved.

At its core, the framework uses a simple pattern:

Data Units → Operations → Data Units

A Thought Tree program applies this recursively. Complex cognitive work is decomposed into named artefacts, transformations, contracts, modules and traces.

The long-term aim is to move LLM-assisted work from isolated prompting toward cognitive programming: reusable, inspectable, model-independent programs whose primary objects are ideas, documents, plans, requirements, reviews, concepts and transformations between them.

Current Status

This is a handoff release.

The Thought Tree Framework is conceptually developed and partially prototyped, but it is not a finished production system.

The original author is stepping back due to family commitments, caring responsibilities, work commitments, burnout recovery and the fact that the project now needs skills, time and expertise beyond one person.

This repository is being released openly so that others can:

- study the idea;
- critique it;
- fork it;
- implement a Cognitive Engine;
- refine TTML;
- build examples;
- compare it with existing agent and workflow frameworks;
- or take the project in a better direction.

The goal of this release is not to claim the framework is complete. The goal is to prevent the idea from being trapped with one person.

The Problem

LLMs are powerful at working with ambiguous human concepts: documents, requirements, notes, stories, plans, reviews, policies, designs and decisions.

But most current LLM use still happens through:

- one-off prompts;
- chat sessions;
- brittle prompt chains;
- loosely controlled agents;
- bespoke scripts;
- workflow automations without semantic validation.

These approaches can be useful, but they often lack qualities expected from production systems:

- explicit structure;
- inspectable intermediate results;
- reusable modules;
- dependency management;
- validation;
- versioning;
- audit trails;
- error handling;
- separation between workflow definition and model provider;
- and clear provenance for final outputs.

For small tasks, a prompt may be enough.

For larger cognitive work, the process matters.

The Core Idea

A Thought Tree program describes cognitive work as transformations over named artefacts.

Inputs



Operations



Intermediate Artefacts



Review / Validation



Final Outputs



Execution Trace

For example, instead of asking an LLM:

"Write a technical design document from these notes."

a Thought Tree module might define:

design_notes



CreateSourceDigest



source_digest



ExtractRequirements



requirements_register



DesignSystemDecomposition



system_decomposition



DraftSections



draft_tdd_sections



AssembleDraft



draft_tdd



ReviewDraft



review_and_correction_plan



ReviseFinalDocument

↓

technical_design_document

Each step produces an artefact that can be inspected, validated, replaced, reused or traced.

Key Concepts

##Data Unit A Data Unit is a discrete artefact used or produced by a workflow.

Examples:

- source document;
- requirements register;
- plot outline;
- chapter draft;
- review report;
- validation report;
- generated TTML module;
- final document.

In the current TTML draft, Data Units are often represented as files, but conceptually they may be any addressable artefact.

Operation

An Operation transforms input Data Units into output Data Units.

Examples:

source_notes → CreateSourceDigest → source_digest

draft_chapter + feedback → ReviseChapter → revised_chapter

sections → ConcatenateFiles → compiled_document

Operations may be executed by:

- an LLM;

- a deterministic function;
- an external tool;
- another module;
- a generated submodule;
- or a human reviewer.

Module

A Module is a reusable cognitive program.

It declares:

- inputs;
- variables;
- iterators;
- collections;
- operations;
- intermediate artefacts;
- final outputs;
- optional contracts and validation expectations.

A Module may run as a standalone workflow or be called by another Module.

Cognitive Engine

A Cognitive Engine is the compiler and runtime for Thought Tree programs.

It is responsible for:

- loading a Thought Tree definition;
- validating its structure;
- resolving variables, inputs, references, collections and iterators;
- compiling the workflow into an executable transformation graph;
- detecting dependency errors and output collisions;
- invoking LLMs, functions, tools, submodules and human review steps;
- storing intermediate artefacts;
- validating outputs;
- recording execution traces;
- and returning final outputs.

The LLM is not the whole system.

In Thought Tree:

- LLMs handle ambiguity.
- Code handles structure.

The Cognitive Engine coordinates both.

TTML

Thought Tree Markup Language, or TTML, is the current draft XML-based source format for Thought Tree Modules.

A minimal TTML-style workflow might look like:

```
<TTML version="0.12.0">
```

```
<Project
```

```
  id="ArticleSummaryModule"
```

```
  desc="Summarise an article through draft, review and revision." />
```

```
<Inputs>
```

```
  <File id="source_article" folder="/inputs" extension="txt"/>
```

```
</Inputs>
```

```
<Operations>
```

```
  <Operation
```

```
    id="DraftSummary"
```

```
    type="TextCompletion"
```

```
    desc="Create a concise summary of the source article.">
```

```
    <FileRef id="source_article"/>
```

```
  <Output>
```

```
    <File id="draft_summary" extension="txt"/>
```

</Output>

</Operation>

<Operation

id="ReviewSummary"

type="TextCompletion"

desc="Review the draft summary against the source article. Identify omissions, inaccuracies and unsupported claims.">

<FileRef id="source_article"/>

<FileRef id="draft_summary"/>

<Output>

<File id="summary_review" extension="txt"/>

</Output>

</Operation>

<Operation

id="ReviseSummary"

type="TextCompletion"

desc="Revise the summary using the review. Produce the final summary only.">

<FileRef id="source_article"/>

<FileRef id="draft_summary"/>

<FileRef id="summary_review"/>

<Output>

<File id="final_summary" extension="txt"/>

</Output>

</Operation>

</Operations>

<Output>

<FileRef id="final_summary"/>

</Output>

</TTML>

TTML is not the entire framework. It is one source representation of the underlying Thought Tree Program Model.

Future source formats could include YAML, JSON, visual graphs or higher-level authoring tools.

How Thought Tree Differs from Other Approaches

Compared with a prompt:

A prompt asks a model for an answer.

A Thought Tree defines the process by which an answer should be produced, reviewed, revised and traced.

Compared with a prompt chain

Prompt chains connect model calls.

Thought Tree formalises:

- named artefacts;
- dependencies;
- operation types;
- intermediate outputs;
- validation;
- execution traces;
- module reuse;
- and model-independent execution.

Compared with autonomous agents

Agents can be flexible, but may be difficult to predict, debug or audit.

Thought Tree favours explicit structure:

defined process → inspectable plan → controlled execution → preserved artefacts → traceable output

Dynamic planning is still possible, but generated plans or submodules should be validated and recorded.

Compared with traditional workflow tools

Traditional workflow tools are good at deterministic processes.

Thought Tree is designed for hybrid cognitive work:

- LLMs for semantic transformation;
- code for deterministic transformation;
- tools for external capabilities;
- humans for judgement and approval;
- a Cognitive Engine for orchestration and traceability.

Example Use Cases

Thought Tree is intended for complex cognitive production tasks where quality, traceability and iteration matter.

Potential use cases include:

- transforming scattered notes into formal documentation;
- generating technical design documents;
- recovering legacy project knowledge;
- producing compliance or audit preparation packs;
- generating and reviewing creative media;
- producing game content and lore;
- drafting and revising long-form fiction;
- extracting requirements from source documents;
- producing recurring research reports;
- generating, validating and improving other Thought Tree Modules.

What Exists Now

This handoff package includes or is intended to include:

- the full Thought Tree Framework explainer;
- draft TTML concepts and examples;
- the Thought Tree Program Model;
- Cognitive Engine architecture notes;
- execution semantics;
- semantic contract concepts;
- authoring guidance;
- example workflows;
- improvement process notes;
- roadmap;
- prototype code.

Two prototype efforts exist:

1. **Early proof of concept** A C#/Unity prototype demonstrated that an LLM workflow could pass text files between operations as variables. This prototype did not use TTML. It used a hardcoded array of prompts. It successfully planned, drafted and reviewed a 50,000-word novel from a 500-word user-submitted description.
2. **Partial Cognitive Engine prototype** A later C#/Unity project began implementing a Cognitive Engine. It can connect to:
 - Anthropic;
 - OpenAI;
 - local LLMs through KoboldCPP.

It supports basic TextCompletion execution. It does not yet include a stable TTML importer, full graph compilation, contracts, validation gates or a production workspace/trace system.

What Is Not Finished

The framework still needs substantial work.

Known gaps include:

- stable TTML schema;
- TTML importer;

- reference Cognitive Engine;
- CLI runner;
- execution graph compiler;
- iterator and collection resolution;
- output collision detection;
- execution trace format;
- semantic contract format;
- validation system;
- module registry;
- conformance tests;
- authoring tools;
- visualisation tools;
- security and governance model;
- production implementation.

This is a foundation, not a finished product.

Suggested Next Steps for Contributors

Useful contribution areas include:

Specification

- refine the Thought Tree Program Model;
- stabilise TTML;
- define conformance levels;
- define execution trace schema;
- define semantic contract schema.

Reference Engine

- implement a minimal CLI Cognitive Engine;
- parse TTML;
- resolve Data Units and FileRefs;
- execute TextCompletion operations;
- execute deterministic functions;
- preserve intermediate artefacts;
- generate an execution trace.

Examples

- create small TTML examples;

- create larger workflow examples;
- create examples for technical documentation, creative writing, compliance, research and module improvement.

Comparison and Research

- compare Thought Tree with LangGraph, AutoGen, CrewAI, Semantic Kernel, workflow engines and agent systems;
- identify where the framework overlaps, differs or can interoperate.

Tooling

- build a TTML linter;
- build a graph visualiser;
- build a visual editor;
- build an authoring assistant;
- build a module test harness.

Minimal Reference Engine Target

A minimal Cognitive Engine should be able to:

- load a TTML file;
- validate its structure;
- resolve root-level inputs;
- execute operations in document order;
- support TextCompletion;
- support registered deterministic functions;
- store intermediate artefacts;
- resolve final outputs;
- record an execution trace.

A more advanced engine could add:

- dependency graph compilation;
- iterators;
- collections;
- submodules;
- dynamic planning;
- semantic contracts;
- human review;
- caching;
- parallel execution;

- module libraries;
- automated improvement workflows.

Repository Structure

thoughttree-framework/ README.md HANDOFF.md STATUS.md ROADMAP.md
CONTRIBUTING.md GOVERNANCE.md LICENSE.md

docs/ ONE_PAGE_OVERVIEW.md WHY_THIS_MATTERS.md ARCHITECTURE.md
PROGRAM_MODEL.md COGNITIVE_ENGINE.md EXECUTION_SEMANTICS.md
SEMANTIC_CONTRACTS.md AUTHORIZING_GUIDE.md ThoughtTreeFramework.pdf

spec/ TTML_DRAFT.md TTMLSchema.xsd EXECUTION_TRACE_SCHEMA_DRAFT.json
CONTRACT_SCHEMA_DRAFT.json

examples/ article-summary-review/ novel-generation-pipeline/ video-game-tdd/
compliance-gap-analysis/ module-improvement/

prototypes/ unity-novel-poc/ unity-cognitive-engine/

Who Might Be Interested?

This project may be relevant to:

- LLM tooling developers;
- AI workflow framework builders;
- agent framework researchers;
- software architects;
- technical writers;
- compliance and audit tooling developers;
- game and narrative tool developers;
- AI safety and governance researchers interested in traceability;
- people building model-independent AI infrastructure;
- people interested in structured human/LLM collaboration.

Author Handoff

The Thought Tree Framework was created by Robert Bateman.

I have taken the Thought Tree Framework as far as I reasonably can.

Due to family commitments, caring responsibilities, work commitments, burnout recovery and the fact that the project now needs skills and networks beyond my own, I am releasing it openly.

My hope is that others can take the idea further: implement engines, refine TTML, compare it with existing workflow and agent frameworks, build examples, create module libraries, or evolve the concept into something better.

I do not want the idea to be trapped with me.

License

Code: MIT. Documentation/specification: Creative Commons Attribution 0 (CC0) Attribution preferred, not required. See LICENSE and LICENSE-DOCS.md.

Short Version

Thought Tree is a proposed framework for cognitive programming.

It treats LLM-assisted work as an executable graph of:

Data Units → Operations → Data Units

A Cognitive Engine compiles and executes that graph using:

- LLMs;
- deterministic functions;
- tools;
- submodules;
- human review;
- validation;
- and execution traces.

The aim is to make complex cognitive work:

- programmable;
- inspectable;
- reusable;
- auditable;
- model-independent;
- validatable;
- and improvable.

This repository is a public handoff of the idea, documentation, draft standard and prototype work.